Empirical Evaluation of a Generate–Validate–Repair Loop for Intent→Network Configuration Using a Deterministic Reachability Validator

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We present a fully preregistered, artifact-archived paired experiment (study cns-m2026-01) that measures the marginal effect of inserting a deterministic reachability validator plus at most one automated repair (guarded GVR loop) around a single LLM generator (declared gpt-5-mini) for intent→device-configuration NetOps tasks. In a shared-initial paired design on 300 intents (600 episodes), baseline (generator candidate) execution-validated correctness = 299/300 (99.667%); guarded pipeline = 300/300 (100.0%). Paired difference = 0.003333; exact McNemar two-sided p = 1.0; 95% paired bootstrap percentile CI = [0.00, 0.01] (10,000 resamples, seed = 1729). Provider-call accounting: 301 model calls (300 generator + 1 repair). We archive all raw outputs, provider traces, validator traces, per-episode scoring JSONs, manifests, and SHA-256 hashes to enable deterministic reproduction. We describe the preregistered design, deterministic scoring semantics, full reconciliation accounting, representative artifact-grounded examples, validator failure taxonomy, and operational implications supported by the archived evidence.**

I. Introduction

Generative LLMs are being proposed for intent-driven NetOps (intent→configuration). Without execution gating, incorrect or unsafe generator outputs can cause unintended reachability or policy changes when applied to devices. A common systems pattern is generate→validate→(repair): a deterministic validator rejects unsafe candidates and optionally triggers repairs. We preregistered (cns-m2026-01) a controlled paired study to quantify the guarded GVR loop's marginal effect on execution-validated correctness using a single declared generator (gpt-5-mini). The study is fully artifact-archived and reproducible; key manifest and hash references appear in the Disclosure Statement. The final research question: Does an LLM-driven GVR loop using a deterministic reachability validator increase the paired rate of execution-validated correct configurations versus plain LLM generation on curated NetOps intent→configuration tasks? Contributions: (1) a preregistered shared-initial paired evaluation with full artifact audit; (2) deterministic validator semantics and per-episode traces; (3) exact provider-call accounting; (4) an evidence-grounded failure taxonomy and operational discussion. All numeric claims here derive from archived artifacts listed below.

II. Related Work

Prior work has explored LLMs for intent translation, verifier-assisted pipelines, and generate–validate–repair patterns in code and engineering domains. We position our study as complementary: rather than an open leaderboard or multi-model sweep, we preregister a single-model, shared-initial paired design and archive every provider call and validator trace to permit deterministic reconciliation. Representative contextual leads used in our literature positioning are preserved in the evidence bundle (see Cited Record Ids). Our approach follows recommended best practices for contamination control, preregistration, and reporting for LLM evaluation in operational settings.

III. Methodology

Preregistration and estimand. The experiment design, inclusion/exclusion rules, metrics, and statistical plan were preregistered (study_id cns-m2026-01; preregistration SHA-256: 0726a6c02c1ed6fc379927f4bb0696330f0b00484f15caee5cf4ebc072316bf7
Primary estimand: paired_success_rate_difference_guarded_minus_baselin
in a shared-initial paired design. N=300 intents stratified by deterministic complexity heuristics; each intent produced one shared initial generator candidate reused by both conditions. Repair policy: at most one automated repair call per intent, as preregistered. Stopping rule: process the full preregistered N unless model-call or wall-clock budget is exhausted. Model and orchestration. Declared generator: gpt-5-mini (execution/model_configuration.json SHA-256: a40e96e2...), provider calls recorded (execution/provider_calls/, representative trace execution/provider_calls/task-000001-shared-initial.json SHA-256: cfd2d095...). Validator. The deterministic validator (deterministic_netops_validator_v5, v5.0) parses candidate configuration, simulates state transitions on an archived emulator snapshot, enforces task constraints (allowed_touched_fields, group constraints, preserve_unspecified_state), and returns a structured pass/fail plus a detailed validator trace. Scoring. Per-intent outcome = 1 iff validator reports execution-validated configuration satisfying intent constraints and no unintended reachability/policy changes. Primary test: two-sided exact McNemar on discordant pair counts; paired bootstrap percentile 95% CI computed with 10,000 resamples (seed = 1729). Reproducibility. All raw responses, per-call provider traces, per-episode scoring JSONs (execution/scoring/), the task manifest (execution/task_manifest.jsonl SHA-256: 91564bb6...), the raw results file (execution/raw_results.jsonl SHA-256: 176776d1...), and reconciliation manifest

(analysis/deterministic_reconciliation.json SHA-256: f6da5050...) are archived.

IV. Results

Execution and reconciliation. Planned episodes = 600 (300 intents × shared initial); completed episodes = 600; technical failures = 0. Model calls used = 301 (300 generator calls + 1 repair call). Complete reconciliation and provider audit are archived (analysis/deterministic_reconciliation.json SHA-256: f6da5050...). Primary outcomes (analysis/condition_summary.csv SHA-256: 55eacb88...): baseline successes = 299/300 (0.996667); guarded successes = 300/300 (1.0). Paired contingency (analysis/paired_contingency_table.csv SHA-256: 3421adb5...): n11 = 299, n10 = 1, n01 = 0, n00 = 0. Observed paired difference (guarded - baseline) = 0.0033333333333332993. Statistical inference: exact McNemar two-sided p = 1.0 for observed discordant counts (implementation archived in analysis artifacts); paired bootstrap percentile 95% CI = [0.00, 0.01] (10,000 resamples; seed = 1729). Repair accounting. A single automated repair call (execution/provider_calls/task-000117-guarded-repair.json SHA-256: 4deb410d...) converted the only baseline failure to success; validator diagnostics show no validator false negatives across the corpus. Representative episodes and their per-episode scoring JSONs are archived (examples: execution/scoring/task-000001-baseline.json SHA-256: 2e4a1ef1... and execution/scoring/task-000003-guarded.json SHA-256: 635b97a3...). Figures and tables summarizing these artifacts are available in the archived analysis bundle.

V. Discussion

Interpretation. On this curated emulator corpus the generator was already near ceiling (baseline 299/300). The guarded GVR loop corrected the single residual error with one automated repair call, demonstrating that, where generator quality is high and validator fidelity is appropriate, a deterministic validator plus a constrained repair budget can eliminate residual execution risks at minimal extra model cost. Statistical nuance. The observed discordant counts are very small (n10=1, n01=0); the exact McNemar two-sided p reported (p=1.0) and the bootstrap CI ([0.00,0.01]) are archived with seeds and resample counts for deterministic reproduction (analysis/analysis_log.jsonl SHA-256: 7db06f53...). Threats to validity. Internal: prompt sensitivity and generator nondeterminism were mitigated by shared-initial semantics and archiving raw outputs; construct: the validator operates on an archived deterministic emulator snapshot and therefore omits production non-determinism (firmware variants, timing effects); external: single declared model tested — no multi-model generality claimed. Operational implications. For deployments where generator accuracy is very high and validator latency is acceptable, a deterministic validator with a small repair budget can function as an effective execution gate. Operators must verify validator fidelity on their live device fleet before deploying such a gate. Reproducibility. All artifacts required to reproduce the run and the analyses (raw responses, scoring JSONs, provider traces, manifests, and SHA-256 hashes) are archived and referenced in the Disclosure Statement.

VI. Limitations

- High baseline ceiling (baseline = 299/300) reduces power to detect small practical effects; we report exact paired counts and a paired bootstrap CI to transparently convey uncertainty.
- Construct validity: the deterministic emulator validator operationalizes safety as absence of emulator-observed unintended reachability/policy changes in a closed snapshot; production non-determinism (firmware variants, timing effects, cross-domain interactions) is outside scope.
- Model scope: single declared model (gpt-5-mini) — no claim of multi-model generality.
- Design semantics: shared-initial paired design isolates the validator marginal effect but does not measure pipelines that requery the generator differently per condition.
- Repair budget: only one automated repair attempt permitted; iterative multi-call repair strategies were not evaluated in this confirmatory run.

VII. Conclusion

We conducted a preregistered, artifact-archived paired evaluation of a deterministic validator + <=1 automated repair (guarded GVR) vs plain LLM generator for 300 intent→configuration tasks. Observed outcomes: baseline 299/300; guarded 300/300; paired difference 0.00333; exact McNemar two-sided p = 1.0; 95% paired bootstrap percentile CI = [0.00,0.01] (10,000 resamples, seed = 1729). Model calls used = 301 (300 generator + 1 repair). The guarded GVR loop eliminated the single residual failure in this controlled emulator corpus at minimal additional model usage. These results are artifact-grounded and reproducible via the archived manifests and SHA-256 hashes. Broader production conclusions require further preregistered evaluations across multiple models, lower baseline accuracy regimes, and live heterogeneous device fleets.

Disclosure Statement

Mandatory Disclosure Statement (artifact references and runtime facts)

Initial master prompt (immutable archive): provenance/master_prompt.txt — SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15
Preregistration: study_id = cns-m2026-01 — preregistration SHA-256: 0726a6c02c1ed6fc379927f4bb0696330f0b00484f15caee5cf4ebc072316bf7 (preregistration file archived with the run).
Declared generator and configuration (archived): execution/model_configuration.json — SHA-256: a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd82
Declared model name: gpt-5-mini (as recorded in execution/model_configuration.json). Provider

family and per-call traces: execution/provider_calls/ (full per-call traces archived; representative shared initial trace execution/provider_calls/task-000001-shared-initial.json — SHA-256: cfd2d0956873ca732f5278c5d9bd64f0ffb008623d7fb52d2958c7769b69a791). Orchestration framework: adapter_family = hosted_netops_gvr_v1 (execution manifest). Deterministic validator: deterministic_netops_validator_v5 (validator v5.0) implemented for parsing, deterministic emulation, constraint checking, and structured trace output. Repair policy: one automated repair attempt permitted per intent when the validator failed (preregistered). Representative repair trace archived: execution/provider_calls/task-000117-guarded-repair.json — SHA-256: 4deb410d5736ab332cd5caaf00b16051e5b0aef2276255c0770098f63cb523ba. Preregistered design freeze: shared-initial paired semantics, task_count = 300, planned_episode_count = 600, maximum_model_calls = 600, repair_budget <=1, exact McNemar two-sided test as primary, paired bootstrap percentile CI (10,000 resamples, seed = 1729) for paired difference. The archived preregistration contains the full sampling, exclusion, retry, and analysis plans (preregistration SHA above). Execution artifacts and hashes (selection): raw results — execution/raw_results.jsonl (SHA-256: 176776d1c015a652ea2d28ced2be39ffbfec9b969b72ab03e1a1193b567b6dbb); task manifest — execution/task_manifest.jsonl (SHA-256: 91564bb69ac692e2fbd6321708863d13938309b5433ef1b211541ba5426df990); paired contingency table — analysis/paired_contingency_table.csv (SHA-256: 3421adb52d5791a979397368165fc2062e6b9a4a771ce49c0bcac80b02215d55); reconciliation — analysis/deterministic_reconciliation.json (SHA-256: f6da505032ab388ceccda8e85084b53b8cada91a7ccc0dc65601385505f00ea6); analysis log — analysis/analysis_log.jsonl (SHA-256: 7db06f53d650b2ffc5838b392740c75681480829e5db575e44a9a065eadb965e). The execution manifest records model_calls_used = 301 (300 generator + 1 repair) and completed_episode_count = 600 (planned = 600), with no technical failures (execution/execution_manifest.json). Human role and autonomy boundary. The Research Director selected the broad topic family (Generative AI / LLMs for NetOps) and approved the initial master prompt prior to the autonomy lock. After the lock the autonomous system performed literature synthesis, research-gap selection, preregistration, code generation, execution, analysis, manuscript drafting, peer review, and revision. No human manually edited the technical manuscript content after the autonomy lock. All manuscript numbers and claims correspond to archived run artifacts cited above. Limitations (abridged): single declared model tested; deterministic emulator validator abstracts production heterogeneity; shared-initial semantics isolate gate marginal effect but do not evaluate requery pipelines; repair budget limited to one automated attempt. See manuscript body for fuller limitations and threats to validity. All artifact paths and SHA-256 values above are copied verbatim from the autonomous run archive and are offered as immutable references for deterministic reproduction.

REFERENCES

[1] https://doi.org/10.1145/3656296
[2] https://doi.org/10.1109/icc59461.2026.11587314
[3] https://doi.org/10.36227/techrxiv.177205043.39321579/v1